

CS455 – Software Engineering Course Project

AI-Assisted Software Development Life Cycle Project

1. Objective

This project takes you through the full Software Development Life Cycle (SDLC) in the era of AI-assisted software engineering. You are expected to use LLMs and AI tools extensively, but the project will not be evaluated primarily on how much code or documentation an AI system can generate. The emphasis is on engineering judgment: requirements engineering, architecture, design, project management, implementation, testing, debugging, security, reliability, deployment, and cost planning.

The project must demonstrate an end-to-end engineering process in which requirements, design decisions, Jira issues, GitHub development activity, tests, defects, and deployed functionality remain traceable.

You will be evaluated on your ability to:

- Perform requirements engineering and manage requirements changes.
- Use Jira for backlog management, sprint planning, task assignment, defect tracking, and project monitoring.
- Use GitHub for collaborative development and integrate it with Jira.
- Design and justify an appropriate architecture and design patterns.
- Use LLMs and agentic AI tools effectively while critically evaluating their outputs.
- Implement, review, test, debug, secure, and deploy a working system.
- Demonstrate concurrency correctness, reliability, performance, and failure handling.
- Maintain end-to-end traceability from requirements to implementation and testing.
- Make and defend engineering decisions rather than merely accepting AI-generated solutions.

2. Choosing Your Project

There is no fixed problem statement. Each group must propose its own project. You are free to pick a domain that interests you, provided the system satisfies the constraints below.

Constraints on Your Choice

- Support **at least two distinct user roles** with different permissions and separate dashboards.
- Involve **a stateful transactional workflow** in which something is created, modified, and cancelled, with consequences for inventory, seats, slots, stock, capacity, or another shared resource.
- Have **shared, contended resources** so that **concurrency correctness** is a real concern and no unit can be allocated twice.

- Include **onboarding and removal of one class of entity**, handled by an administrator or the platform itself.
- Contain **at least one rule-driven computation** such as pricing, scoring, ranking, scheduling, or allocation that is non-trivial and testable.
- Have meaningful scope for **agentic AI integration**. The agent must perform a useful task within the workflow; a chatbot bolted onto the application is not sufficient.
- Be sufficiently complex to support requirements evolution, testing, security analysis, performance evaluation, and failure-handling experiments.

Not Permitted

- Static or purely informational websites.
- Single-role CRUD applications with no contention and no meaningful workflow.
- Re-submission of an existing project from another course, internship, or hackathon.
- Projects whose main functionality is simply an LLM chatbot without a substantive software system around it.

3. Project Proposal

Before starting implementation, each group must submit a one-page proposal containing:

- Project title and a short description of three to five lines.
- The user roles and what each role can do.
- The core transactional workflow.
- The shared/contended resource and the concurrency challenge.
- Where you intend to place AI agents, what tools they may use, and why.
- The proposed technology stack.
- Initial project risks and major technical uncertainties.
- Initial Jira/GitHub development strategy and the links. Also add the instructor and the TAs to both Jira Github project.

Proposals will be approved, returned for revision, or rejected. Do not begin Deliverable 1 before your proposal is approved.

4. Jira + GitHub Integrated Project Management

Jira and GitHub are mandatory components of the project. Jira must be used to manage requirements, user stories, tasks, sprints, bugs, priorities, and progress. GitHub must be used for source-code management and collaborative development.

Jira Requirements

- Create a Jira project accessible to the instructor/TA.
- Represent major functional and non-functional requirements as Jira issues.
- Maintain a product backlog and sprint backlog.
- Create user stories, tasks, and sub-tasks and assign them to individual students.
- Track bugs and defects in Jira.

- Use priorities, status, labels/components, estimates, and appropriate issue relationships.
- Maintain a dashboard showing project progress.

Mandatory Jira–GitHub Integration

Teams must configure Jira–GitHub integration and maintain traceability throughout development. For significant work, the expected chain is:

Requirement → User Story → Jira Issue → GitHub Branch → Commit(s) → Pull Request → Test Case → Test Result

- Jira issue identifiers must be used consistently in relevant GitHub branches, commits, and/or pull requests.
- Significant features must be developed through branches and integrated using pull requests or an equivalent reviewed workflow.
- Pull requests must contain meaningful descriptions, testing information, and the relevant Jira issue.
- At least one other team member must review significant pull requests.
- Jira issues must be updated as work progresses; retrospective creation of issues immediately before submission will not receive full credit.
- GitHub commits and Jira activity will be considered jointly when evaluating individual contribution.

Sprint-Based Development

- Define a sprint goal.
- Select and estimate work from the backlog.
- Assign work to individual members.
- Track completed, incomplete, and carried-forward work.
- Conduct sprint review and retrospective activities.
- Maintain evidence of development throughout the semester.

5. Requirements Engineering and Traceability

Each team must elicit, specify, justify, and manage its own functional and non-functional requirements.

- Requirements must be unambiguous, verifiable, and testable.
- Every major requirement must map to one or more use cases/user stories and Jira issues.
- Mandatory and optional requirements must be clearly distinguished.
- AI-agent behaviour must be specified as requirements, including what an agent may decide autonomously, what requires human approval, and what the agent must never do.
- Requirements must be reviewed with an LLM, but students must document what the LLM changed and what recommendations were rejected.
- Maintain an end-to-end traceability matrix covering critical requirements.

For at least 8–10 critical requirements, demonstrate: Requirement → Jira issue → Implementation/PR → Test case → Test result.

6. Requirements Change and Change Impact Analysis

During the project, the instructor/TA may introduce one or more change requests after the initial requirements deliverable. Teams must treat these as genuine requirements changes rather than simply adding an extra feature.

- Record the change request in Jira.
- Assess its impact on requirements, architecture, implementation, testing, schedule, and cost.
- Update affected artifacts.
- Update the backlog and sprint plan.
- Implement the approved change.
- Add or modify tests.
- Document the final impact and engineering trade-offs.

7. Architecture & Design

- Use an LLM to assist with architecture generation, but critically evaluate the proposed architecture.
- Document at least two important alternatives considered and rejected.
- Provide system architecture, component, and sequence diagrams.
- Identify and apply relevant design patterns and justify their use.
- Document at least three significant engineering decisions, including alternatives, trade-offs, and rationale.
- At least three significant LLM-generated recommendations must be explicitly evaluated and, where appropriate, rejected with justification.

The architecture must address security, concurrency, failure handling, observability, scalability, and AI-agent boundaries where relevant.

8. AI-Assisted Development and AI Engineering Log

LLMs may be used extensively for requirements, architecture, design, implementation, testing, debugging, documentation, and planning. However, students remain responsible for every submitted artifact.

For major AI-assisted artifacts, maintain an AI Engineering Log containing:

- Prompt or interaction record.
- LLM output.
- Student evaluation of the output.
- Errors, risks, hallucinations, or inappropriate recommendations identified.
- Changes made by the students.

- Final accepted artifact.
- Reason for accepting, modifying, or rejecting the LLM output.

Prompt histories alone are insufficient evidence of engineering work.

9. Agentic AI Integration and Governance

The project must include at least one meaningful agentic AI workflow.

- Define the agent's purpose and responsibilities.
- Specify tools/APIs the agent can invoke.
- Define autonomous actions, approval-required actions, and prohibited actions.
- Implement validation or policy checks around consequential actions.
- Test incorrect tool selection, invalid outputs, hallucinations, and other relevant failure modes.
- Maintain an audit trail of important agent decisions and tool invocations.

Students must identify at least three failure or risk scenarios for their agent and implement appropriate safeguards.

10. Implementation and Code Review

- Use GitHub as the primary source-code repository.
- Use individual student accounts; account sharing is not permitted.
- Use a documented branching strategy.
- Use pull requests for significant changes.
- Require meaningful peer code review.
- Link significant implementation work to Jira issues.
- Maintain automated builds/tests where practical.
- Document significant code-generation assistance received from AI tools.

11. Testing, Adversarial Testing and Debugging

AI-generated tests are encouraged but must be critically reviewed and supplemented by student-designed tests.

- Unit testing.
- Integration testing.
- System/end-to-end testing.
- Adversarial testing designed to break the system.
- Regression testing for every significant defect fixed.
- Minimum 80% code coverage unless a justified exception is approved.
- Concurrency testing for shared resources.
- Failure and recovery testing.
- Security testing.
- Performance/load testing.

Each team must deliberately discover and fix at least five meaningful bugs or weaknesses. At least two should be found through adversarial, concurrency, security, or failure-oriented testing.

12. Concurrency and Reliability Challenge

Teams must construct a reproducible experiment in which multiple users simultaneously attempt to access or allocate the same shared resource.

- Demonstrate the correctness requirement.
- Explain possible race conditions.
- Demonstrate the implemented solution under concurrent access.
- Explain why the chosen synchronization/transaction strategy works.
- Discuss alternatives and their trade-offs.

Teams must also demonstrate system behaviour under at least three failure scenarios, such as database failure, external API/LLM failure, timeout, duplicate transaction, invalid agent output, or server restart.

13. Security Engineering

Each project must include a basic security analysis appropriate to its application.

- Authentication and authorization.
- Role-based access control where applicable.
- Input validation and secure error handling.
- Secure management of secrets and credentials.
- Threat model covering at least five relevant threats.
- At least three security test scenarios.
- At least one authorization-abuse scenario demonstrated and prevented.

14. Performance Engineering

The system must be evaluated under a realistic workload. The existing baseline expectation is at least 100 daily active users and 250+ core transactions per day, or a justified workload appropriate to the selected system.

- Define measurable performance targets.
- Conduct a reproducible load/performance experiment.
- Measure response latency and throughput.
- Identify at least one performance bottleneck.
- Perform at least one optimization.
- Compare measurements before and after optimization.

15. Observability and Auditability

- Implement meaningful application logging.
- Record important business transactions.
- Maintain an audit trail for consequential actions.
- Log relevant AI-agent actions and tool calls.
- Provide enough diagnostic information to investigate failures without exposing sensitive information.

16. Cost Estimation, Project Planning and Deployment

- Use AI to estimate development time and resource requirements.
- Estimate infrastructure, storage, APIs, model inference, and deployment costs.
- Provide a Gantt chart or timeline.
- Deploy the platform on a cloud service such as AWS or GCP.
- Do not exhaust free-tier limits.
- Compare estimated and actual resource/cost usage where possible.

17. Deliverables

All deliverables must be uploaded to the Hello IITK assignment submission link. There will be separate submission link for each deliverable on Hello IITK platform. Also, upload these files to your GitHub project repository in the Deliverables folder. The Jira project must remain available to the instructor/TA.

Deliverable 0 Project Proposal – Due August 29, 2026, 11:59 PM

- One-page proposal as specified in Section 3. Approval is required before further work.

Deliverable 1 – Requirements, Architecture and Project Setup – Due September 26, 2026 11:59 PM

- Requirements specification.
- Use-case diagram and user stories.
- Functional and non-functional requirements.
- Initial requirements traceability matrix.
- Architecture and design document.
- Component/sequence diagrams.
- Design-pattern mapping.
- Architecture alternatives and rejected decisions.
- Jira project with initial backlog.
- Jira–GitHub integration configured.
- Initial sprint plan.
- AI Engineering Log for requirements and architecture activities.

Deliverable 2 – Implementation, Agentic AI and Sprint Execution – Due October 15, 2026 11:59 PM

- GitHub repository with working implementation.
- Agentic AI integration.
- README and API documentation.
- Prompt/AI Engineering Log.
- Jira sprint board and development history.
- Jira issues linked to GitHub branches/commits/pull requests.
- Evidence of peer code reviews.
- Updated requirements traceability.
- Demonstration of the core transactional workflow.

Deliverable 3 – Testing, Security, Performance, Cost, Deployment and Reliability – Due November 6, 2026 11:59 PM

- Unit, integration, system and adversarial test reports.
- Code coverage report.
- Concurrency test report.
- Security/threat-model report.
- Performance/load test report and optimization results.
- Failure/recovery test report.
- Bug/defect history in Jira.
- Evidence of at least five meaningful defects/weaknesses discovered and fixed.
- Costing document.
- Timeline/Gantt chart.
- Deployment documentation.
- Final requirements-to-Jira-to-GitHub-to-testing traceability matrix.
- Final Jira dashboard and project report.
- AI Engineering Log.

Final Project Presentation and Scenario-Based Demonstration – [November 8-13, 2026]

Time slots will be assigned later.

The final demonstration must be scenario-driven rather than a sequence of slides. Each team must demonstrate the normal workflow, at least one concurrent-access scenario, one failure scenario, one security scenario, and the agentic AI workflow.

During the demonstration, teams may be asked to navigate from a requirement to its Jira issue, GitHub implementation/PR, and corresponding test.

18. Individual Viva

Every student will participate in an individual technical viva. The viva will be used to validate individual contribution and understanding.

- Explain your own implementation and design decisions.
- Explain at least one bug or difficult engineering problem you worked on.
- Demonstrate the relationship between your Jira work and GitHub contribution.
- Explain an LLM-generated recommendation that you accepted or rejected.
- Answer questions about testing, security, concurrency, and failure handling relevant to your contribution.

The instructor/TAs may ask a student to explain or modify a part of the system without assistance from an LLM.

19. Engineering Retrospective

Each team must submit a short retrospective addressing:

- What went wrong during development?
- Which requirements changed and why?
- Which architecture or design decision would you change?
- What did the LLM get wrong?
- Where did AI substantially improve productivity?
- Where did AI introduce risk or additional work?
- What was the most difficult bug or failure?
- How did Jira and GitHub support the SDLC?
- What would you do differently in a real production project?

20. Marking Scheme

Component	Marks	Key evaluation focus
Deliverable 1: Requirements + Architecture + Jira setup	20	Requirements quality, traceability, architecture, design decisions, Jira backlog and integration
Deliverable 2: Implementation + Agentic AI + Sprint execution	25	Working system, AI integration, GitHub workflow, PR/code review, Jira execution, engineering quality
Deliverable 3: Testing + Security + Performance + Deployment	25	Testing depth, adversarial testing, concurrency, security, performance, reliability, deployment
Final scenario-based presentation/demo	10	System behaviour, engineering demonstration, Jira/GitHub traceability, handling of failure/security scenarios
Individual technical viva	10	Individual understanding, contribution, debugging and engineering judgment

Engineering retrospective + final documentation	10	Critical reflection, AI evaluation, traceability and documentation
---	----	--

Total: 100 marks. The course project contributes 40% of the overall course grade.

Bonus marks may be awarded for substantial functionality, stronger performance, additional justified non-functional requirements, or exceptional engineering practices. Bonus work must not substitute for the mandatory baseline.

21. Github Project Repository

The team leader should create the GitHub repository, add all team members using their own accounts, and add the instructor and assigned TAs with appropriate access. The team must also create the Jira project and provide access to the instructor and TAs.

All team members must make meaningful contributions from their own GitHub accounts. Contributions made using another student's account may not be counted as individual contribution.

22. Important Principles

- LLM use is encouraged and expected.
- Students remain responsible for all submitted artifacts and software behaviour.
- AI-generated output is not evidence of correctness.
- Jira must reflect the actual project process.
- GitHub history must reflect actual development.
- Requirements, implementation, testing, and documentation must remain consistent.
- Engineering judgment, critical evaluation, debugging, testing, and system understanding are more important than the amount of AI-generated code.
- Artificial commits, artificial Jira activity, fabricated test results, or other attempts to manufacture evidence of work will not receive credit.